torch.cond#

https://pytorch.org/docs/stable/generated/torch.cond.html

Description:

Conditionally applies true_fn or false_fn. torch.cond is a prototype feature in PyTorch. It has limited support for input and output types and doesn't support training currently. Please look forward to a more stable implementation in a future version of PyTorch. Read more about feature classification at: https://pytorch.org/blog/pytorch-feature-classification-changes/#prototype cond is structured control flow operator. That is, it is like a Python if-statement, but has restrictions on true_fn, false_fn, and operands that enable it to be capturable using torch.compile and torch.export. Assuming the constraints on cond's arguments are met, cond is equivalent to the following:

Function Signature

```
torch.cond(pred, true_fn, false_fn, operands=())[source]#
Parameters
Return type
Restrictions:
```

Returns:

Any

Code Examples

```
def cond(pred, true_branch, false_branch, operands):
    if pred:
        return true_branch(*operands)
    else:
        return false_branch(*operands)
```

```
def true_fn(x: torch.Tensor):
    return x.cos()


def false_fn(x: torch.Tensor):
    return x.sin()


return cond(x.shape[0] > 4, true_fn, false_fn, (x,))
```

Notes & Warnings

⚠ **WARNING** Warning torch.cond is a prototype feature in PyTorch. It has limited support for input and output types and doesn't support training currently. Please look forward to a more stable implementation in a future version of PyTorch. Read more about feature classification at: https://pytorch.org/blog/pytorch-feature-classification-changes/#prototype

Detailed Documentation

Documentation

Conditionally applies true_fn or false_fn.

torch.cond is a prototype feature in PyTorch. It has limited support for input and output types and doesn't support training currently. Please look forward to a more stable implementation in a future version of PyTorch. Read more about feature classification at: https://pytorch.org/blog/pytorch-feature-classification-changes/#prototype

cond is structured control flow operator. That is, it is like a Python if-statement, but has restrictions on true_fn, false_fn, and operands that enable it to be capturable using torch.compile and torch.export.

Assuming the constraints on cond's arguments are met, cond is equivalent to the following:

pred (Union[bool, torch.Tensor]) – A boolean expression or a tensor with one element, indicating which branch function to apply.

true_fn (Callable) – A callable function (a -> b) that is within the scope that is being traced.

false_fn (Callable) – A callable function (a -> b) that is within the scope that is being traced. The true branch and false branch must have consistent input and outputs, meaning the inputs have to be the same, and the outputs have to be the same type and shape. Int output is also allowed. We'll make the output dynamic by turning it into a symint.

operands (Tuple of possibly nested dict/list/tuple of torch.Tensor) – A tuple of inputs to the true/false functions. It can be empty if true_fn/false_fn doesn't require input. Defaults to ().

The conditional statement (aka pred) must meet one of the following constraints:

It's a torch.Tensor with only one element, and torch.bool dtype

It's a boolean expression, e.g. x.shape[0] > 10 or x.dim() > 1 and x.shape[1] > 10

The branch function (aka true_fn/false_fn) must meet all of the following constraints:

The function signature must match with operands.

The function must return a tensor with the same metadata, e.g. shape, dtype, etc.

The function cannot have in-place mutations on inputs or global variables. (Note: in-place tensor operations such as add_ for intermediate results are allowed in a branch)